DESIGN AND DEVELOPMENT OF AUTONOMOUS INSPECTION ROBOTIC SYSTEM FOR REAL-TIME DETECTION OF METAL FAILURE USING DEEP LEARNING

A thesis submitted by

Shahmir (GL) **(20MTE034)**

Abdul Qayoom **(20MTE019)**

Kashif Ali **(20MTE026)**

Supervisor

Engr. Aamir Shaikh

Submitted in the partial fulfillment of the requirements for the degree of

Bachelor of Mechatronic Engineering

Faculty of Engineering

DEPARTMENT OF MECHATRONIC ENGINEERING
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO

December, 2024

DEPARTMENT OF MECHATRONIC ENGINEERING

CERTIFICATE

This is to certify that "**PROJECT/THESIS REPORT ON, DESIGN AND DEVELOPMENT OF AUTONOMOUS INSPECTION ROBOTIC SYSTEM FOR REAL-TIME DETECTION OF METAL FAILURE USING DEEP LEARNING"** is submitted in partial fulfillment of the requirement for the degree of Bachelor of Mechatronic Engineering by the following students:

Shahmir (GL)	**(20MTE034)**
Abdul Qayoom	**(20MTE019)**
Kashif Ali	**(20MTE026)**

_____

Supervisor
Engr. Aamir Shaikh

_____ _____

External Examiner **Chairman.**

Date: _____

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to the following people and organizations that have played a major role in the completion of this Project/Thesis:

My Thesis Supervisor, [Engr. Aamir Shaikh]: I am truly grateful for his constant support and guidance, which provided valuable insights throughout the whole process. His expertise has been key in helping me complete and shape this thesis.

Syed Muhammad Haris [KINPOE Faculty]: I would like to express my sincere gratitude to him for his invaluable guidance throughout the project journey, especially in the area of Human-Machine Interface (HMI).

Institutional Support: I would like to thank **[Mehran University of Engineering & Technology, Jamshoro]** for providing me with the necessary resources, laboratory facilities, library facilities and a conducive learning and research environment which played a major role in completing this Project/Thesis.

My Family:A big thank you to my parents and siblings for their support, love, and encouragement throughout the project. Your belief in me has motivated me to keep going in my academic journey.

Research Participants: To all my friends and especially my group members who have participated in this project and research. A deep thank you for your time and interest in contributing some valuable insights, without which this study might not have been possible.

DEDICATION

This Thesis is a special part of my academic journey, and this hard work is dedicated solely to:

My Family: This thesis is dedicated to my family for their unconditional love, constant support, and sacrifices. Your love and sacrifices have been my foundation, and this achievement is equally yours as it is mine.

My Friends: To those friends who stood by with me in my hard times, offered a listening ear, helping hand and shared both the triumphs and struggles.

Table of Contents

INTRODUCTION **11**
1.1 Background 11
1.2 Problem Statement 11
1.3 Aim 12
1.4 Motivation 12
1.5 Objectives 12
LITERATURE REVIEW **13**
2.1 Introduction 13
2.2 Research Papers Overview: 13
2.2.1 Path Planning 13
2.2.2 Deep Learning 13
2.2.3 Robotics in Hazardous Environments 14
2.2.4 Simulation and Software Development 14
2.2.5 HMI and Interface Development: 15
Design and Development of CAD Models **16**
3.1. Introduction: 16
3.1.1 Circular Inspection Robot Design 16
3.1.2 Locomotion System and Chassis 16
3.2. CAD Model of the Inspection Arena 19
3.2.1 Key Components of the Inspection Arena 19
3.2.2 Design Features of the Inspection Arena 21
3.3. Integration of Robot and Arena Design 22
3.4. Conclusion 22
Path Planning and Simulation Using MATLAB **23**
4.1. Introduction 23
4.2. Binary Occupancy Map of the Arena 23
4.2.1 Occupancy Map Creation 23
4.2.2 Code Summary for Occupancy Map 24
4.3. Path Planning Using PRM 25
4.3.1 Code Summary for Path Planning 26
4.4. Robot Simulation 26
4.4.1 Code Summary for Robot Simulation 27
4.5. Results and Discussion 27
4.6. Conclusion 27
Human-Machine Interface for Robot Control **28**
5.1. Introduction 28
5.2. Objective 28
5.3 Login Page 29
5.3.1 User Interface Design 29

5.3.2 Home Page 30
5.3.3 Manual Control Page 32
5.3.4 Simulation Page 33
5.3.5 Autonomous Operation Page 35
5.3.6 Automation Page Features 35
5.4. Real-Time Interaction and Monitoring 36
5.5. Conclusion 37
Metal Crack Detection Using Deep Learning Techniques 38
6.1. Introduction 38
6.2. Dataset Preparation 38
6.2.1 Data Collection 38
6.2.2 Data Cleaning 38
6.2.3 Data Augmentation 39
6.2.4 Data Annotation 39
6.2.5 Data Splitting 40
6.3. Model Training 40
6.3.1 Model Overview 40
6.3.2 Training Pipeline 41
6.4. Model Evaluation 41
6.4.1 Metrics 41
6.4.2 Performance Comparison 42
6.4.3 Key Insights 42
6.4.3 Results 42
6.5. Conclusion 43
System Integration With Hardware And HMI 44
7.1 Introduction 44
7.2 Hardware Integration 44
7.2.1 Logic Overview 44
7.3 Integration of Deep Learning Model 44
7.3.1 ONNX 44
7.3.2 Logic Overview 45
7.4 Live Video Processing with Python 46
7.4.1 Logic Overview 46
7.5 MATLAB Integration 46
7.5.1 Logic Overview 46
7.6 System Flow and Interaction 46
7.7 Conclusion 47
Conclusion and Future Work 48
8.1 Conclusion 48
8.2 Future Work 48
APPENDIX-A: FRONT COVER AND EDGE 50

REFERENCES **51**

List of Figures

Figure 3.1: Complete Assembly of the Robot 17
Figure 3.2: Manipulator Design 17
Figure 3.3: Close-Up of Rack and Pinion Mechanism 18
Figure 3.4: 360-Degree Rotation of the 4-Bar Mechanism 18
Figure 3.5: Retracting and Extending the 4-Bar Mechanism 19
Figure 3.6: CAD Model of RPV Head on Support Blocks 20
Figure 3.7: Close-Up of Support Block 20
Figure 3.8: Radiation Shields with Robot Entry Point 21
Figure 4.1. Binary and Inflated Occupancy Maps of the Robot Environment 24
Figure 4.2. Optimized Path from Start to Goal Using PRM 25
Figure 4.3: Robot at Start and Goal Positions in the Path Planning Simulation 26
Figure 5.1: Login Page 29
Figure 5.2: Navigation Bar 30
Figure 5.3: Image Detection on Home Page 31
Figure 5.4: Live Video Detection on Home Page 32
Figure 5.5: Manual Page Control 33
Figure 5.6: Simulation Page Control 34
Figure 5.7: Autonomous Page Control 35
Figure 5.8: Autonomous Page Task management 36
Figure 6.1: Data Augmentation Visualization 39
Figure 6.2: Data Annotation Visualization 39
Figure 6.3: YOLOv11-seg Model Architecture 40
Figure 6.4: Model Results and Evaluation Metrics 41
Figure 6.5: Models Evaluation And Comparison Results 42
Figure 6.6: Model Predictions 43
Figure 6.7: Confusion Matrix 43
Figure 7.1: ONNX Format Conversion 45

List of Abbreviations

DOF: Degree of Freedom

AMP: Automatic Mixed Precision

mAP: Mean Average Precision

IoU: Intersection over Union

CRUD: Create, Read, Update, Delete

YOLO: You Only Look Once

tflite: TensorFlow Lite

ONNX: Open Neural Network Exchange

PRM: Probabilistic Roadmap

RPV: Remotely Piloted Vehicle

HMI: Human-Machine Interface

WPF: Windows Presentation Foundation

CAD: Computer-Aided Design

mm: Millimeter

2D: Two-Dimensional

CVAT: Computer Vision Annotation Tool

B: Byte

M: Meter

ABSTRACT

In hazardous environments like nuclear reactors, inspecting metal surfaces for cracks is crucial to ensure safe and efficient operation. Manual inspections are time-consuming, complex, and expose workers to risks. This thesis focuses on designing and developing an autonomous inspection robot system that can detect metal cracks in real-time using deep learning.

The system includes a small mobile robot with a 4-degree-of-freedom manipulator to explore confined spaces. It uses deep learning models, like YOLO, to identify cracks accurately. A Human-Machine Interface (HMI) is also developed to allow operators to control the robot and monitor inspections efficiently. The HMI is integrated with MATLAB for path planning and simulation, and the trained deep learning model detects cracks in both live and recorded videos.

This project demonstrates a reliable and efficient solution that automates the inspection of Reactor Pressure Vessels (RPVs), improving safety and reducing costs in critical industrial environments.

Chapter 1

INTRODUCTION

In hazardous environments, such as nuclear reactors, detecting faults on material surfaces, like metal cracks, is essential for ensuring the safe and efficient operation of industrial systems. Manual inspections are complex and time-consuming and expose workers to unsafe conditions. The basis of this thesis is to design and develop an autonomous inspection robotic system with a real-time detection of metal failures using deep learning techniques.

The system combines a small mobile robot, machine learning-based self-driving and crack detection module, and an intuitive Human-Machine Interface (HMI). - This innovation combines hardware design, quality algorithms, and control logic that combines safety, reliability, and efficiency revolutionizing the inspection process.

1.1 Background

Undetected metal failures lead to accidents, so inspection of reactor pressure vessels (RPVs) and similar industrial structures is essential. The mainstream approaches to evaluate such an immense amount of data are labor intensive, requiring not only considerable human resources but also subject matter experts, high-end computer systems, making it inefficient and adding operational risk. These challenges can be met with a transformative solution: robotics and artificial intelligence (AI).

This thesis is based on existing advancements in robotics and deep learning. It uses the circular robot with a 4-degree-of-freedom manipulator to conduct the exploration of narrow areas with the assistance of a CAD-modeled inspection arena that mimics the environments associated with reactors. For instance, deep learning models like YOLO improve the accuracy of the system in detecting even the smallest cracks. Designed and engineered as a next-generation app using modern frameworks, the HMI streamlines control and monitoring to create a more efficient bridge between the latest technology advancement and the operator human experience.

1.2 Problem Statement

Manual inspection of metal surfaces in critical areas like nuclear reactors is slow, risky, inefficient, and expensive.

1.3 AIM

We aim to create an autonomous robotic system that detects metal surface cracks in real-time using robotics and AI, improving inspection safety, efficiency, cost-effectiveness, and reliability.

1.4 Motivation

KANUPP currently spends a lot of money on annual RPV head inspections done by Chinese specialists. By developing an autonomous robotic system, KINPOE can help reduce these costs and improve safety, efficiency, and reliability.

1.5 Objectives

- Design and develop the CAD model of the prototype.
- Perform path planning and simulation in a virtual environment.
- Develop the Human-Machine Interface (HMI).
- Acquire and process data to train a deep learning model for detecting metal failures.
- Integrate the system with the HMI and hardware, and evaluate its performance.

CHAPTER-2

LITERATURE REVIEW

2.1 INTRODUCTION

In recent years, there has been a lot of progress in inspection robots for hazardous environments. Research is focused on improving their design, control, and performance. This includes making better Human-Machine Interfaces (HMI), improving path planning, and using deep learning techniques.

In this Literature Review section, we summarize the important contributions that have been made in these areas to improve inspection robots for hazardous environments.

2.2 Research Papers Overview:

2.2.1 Path Planning:

1. **Path planning implementation using MATLAB**
    - Abbadi and Matousek review two path planning algorithms: RRT and Cell Decomposition. The integration of these methods improves path efficiency, implemented in MATLAB.
2. **Time efficient A algorithm for path planning**
    - A modified A* algorithm for robot path planning that reduces processing time by up to 95% with minimal increase in path length, with simulations in MATLAB.
3. **Path planning for the mobile robot**
    - Han-ye Zhang, Wei-ming Lin, and Ai-xia Chen review mobile robot path planning techniques, focusing on global and local strategies like GA, PSO, APF, and ACO.

2.2.2 Deep Learning:

1. **Weld defect detection and classification based on Deep learning method**
    - Tito Wahyu Purnomo, Finkan Danitasari, and Djati Handoko discuss deep learning techniques like CNNs for weld defect detection and classification.

2. **Deep Learning for Smart Industry: Efficient Manufacture Inspection System With Fog Computing**
    - A paper discussing smart industry and automation in manufacturing using CNNs for defect detection, with real-time processing through fog computing.
3. **Disease detection on the leaves of the tomato plants by using deep learning**
    - This paper explores using deep learning, specifically CNNs, for disease detection on tomato plant leaves.

2.2.3 Robotics in Hazardous Environments:

1. **Robotic Development for the Nuclear Environment**
    - This paper discusses the challenges of developing robots for nuclear environments, including radiation damage and remote control issues, and solutions for safer operation in such environments.
2. **Significant Corrosion of the Davis-Besse Nuclear Reactor Pressure Vessel Head**
    - A paper discussing concerns about corrosion in nuclear reactors, emphasizing the importance of corrosion monitoring and mitigation strategies for safety.

2.2.4 Simulation and Software Development:

1. **A Matlab-based interactive Simulator for mobile robotics**
    - This paper presents a Matlab-based interactive simulator for teaching mobile robotics, focusing on modeling, path planning, and motion control.
2. **Hybrid Programming with C#.NET and MATLAB**
    - A paper on combining C#.NET and MATLAB to streamline application development, focusing on their integration for scientific and engineering projects.
3. **Serial Communication using WPF, RS232 and PIC Communication**
    - This paper discusses serial communication between a WPF application and PIC microcontrollers, with a focus on efficient data handling and thread-safe UI updates.
4. **Implementation of Smart Home Control by Using Low Cost Arduino & Android Design**
    - A paper that discusses a cost-effective home automation system using Arduino and Android, focusing on intelligent energy management.

2.2.5 HMI and Interface Development:

1. **The article walks you through building a Human-Machine Interface (HMI) with C# and WPF**
    - This article explains how to build an HMI for connecting to Siemens S7 PLCs using the MVVM design pattern.
2. **.NET WPF HMI product for building Human-Machine Interfaces**
    - An article that discusses using .NET WPF for HMI, focusing on features like live data communication and ready-to-use controls.
3. **Embedding MATLAB figures in a modern .NET UI (WPF, WinUI, or MAUI)**
    - This article explains how to embed MATLAB figures into .NET user interfaces using MATLAB Compiler SDK to integrate MATLAB's plotting tools into WPF applications.

CHAPTER-3

DESIGN AND DEVELOPMENT OF CAD MODELS

3.1. INTRODUCTION:

This chapter outlines the design and function of a custom-built circular inspection robot, and its environment, a CAD-modeled inspection arena developed specifically for Reactor Pressure Vessel (RPV) head inspection. RPV heads are critical components of nuclear power plants, and it's important to inspect them rigorously to make sure that they are safe and in good condition. This contributes to the inspection of RPV adapters while minimizing human exposure to hazardous environments and reducing the risk of accidents, with a compact and efficient robot design and a well-defined inspection arena.

The robot is equipped with a 4-degree-of-freedom (DOF) manipulator and a 4-bar mechanism that allows the camera to be positioned with precision. It also has a built-in system for movement and protection. The inspection arena, designed in CAD, ensures a safe and stable environment for the robot. It includes support blocks, radiation shields, and a layout that makes it easy for the robot to move around and access the RPV head.

In this chapter, we will go over the robot's mechanical design, its motion control systems, and the layout of the inspection arena, showing how these elements work together to carry out thorough RPV head inspections.

3.1.1 Circular Inspection Robot Design:

The circular inspection robot is designed to navigate confined spaces and conduct detailed inspections in challenging environments. With a diameter of 18 inches (450 mm), the robot’s compact structure ensures maneuverability under the RPV head while maintaining stability during operation.

3.1.2 Locomotion System and Chassis

The robot’s locomotion system consists of two primary wheels, each with a 3-inch diameter, powered by stepper motors for precise movement. Four caster wheels—two at the front and two at the back—enhance stability. A protective cover shields the internal components from harsh environmental conditions, ensuring reliability during operation.

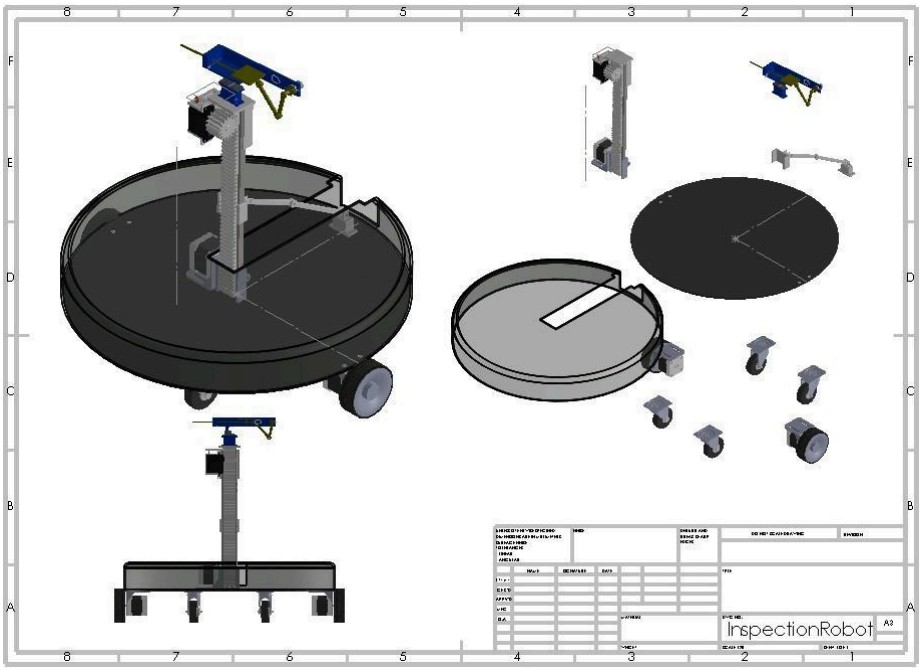


Figure 3.1: Complete Assembly of the Robot

3.1.3 Integrated 4-DOF Manipulator

At the core of the robot's functionality is a 4-DOF manipulator mounted at the center of the chassis. This manipulator enables the robot to perform complex inspection tasks with high precision. The four degrees of freedom include:

1. **1st DOF: Stepper Motor Actuation:**
   The first degree of freedom allows the manipulator arm to rotate from a **horizontal** (0°) position to a **vertical** (90°) position using a **stepper motor**. This feature enables the robot to fold the manipulator when entering tight spaces and deploy it once inside, ensuring the manipulator's **compact storage and efficient deployment**.

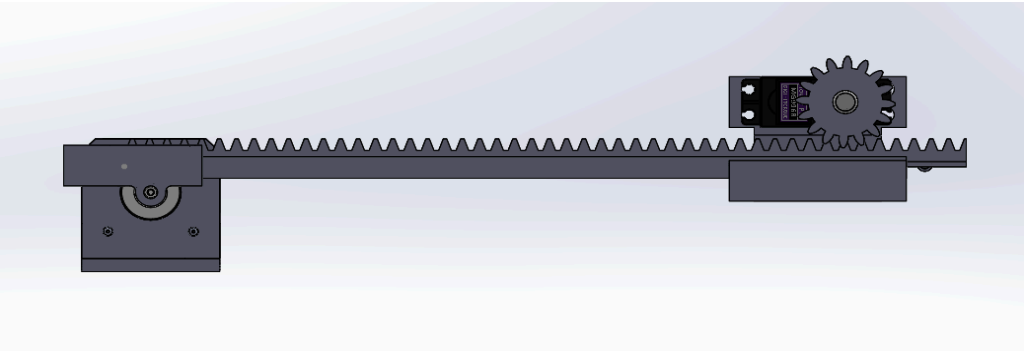


Figure 3.2: Manipulator Design

2. **2nd DOF: Rack and Pinion Mechanism**
   The second degree of freedom utilizes a **rack and pinion mechanism** to extend the manipulator outward. The **rack** has a total length of **305 mm**, and the mechanism is driven by a **servo motor**. This allows the manipulator to extend and retract smoothly, providing additional reach when needed. The total stroke length for this extension is **270 mm**.

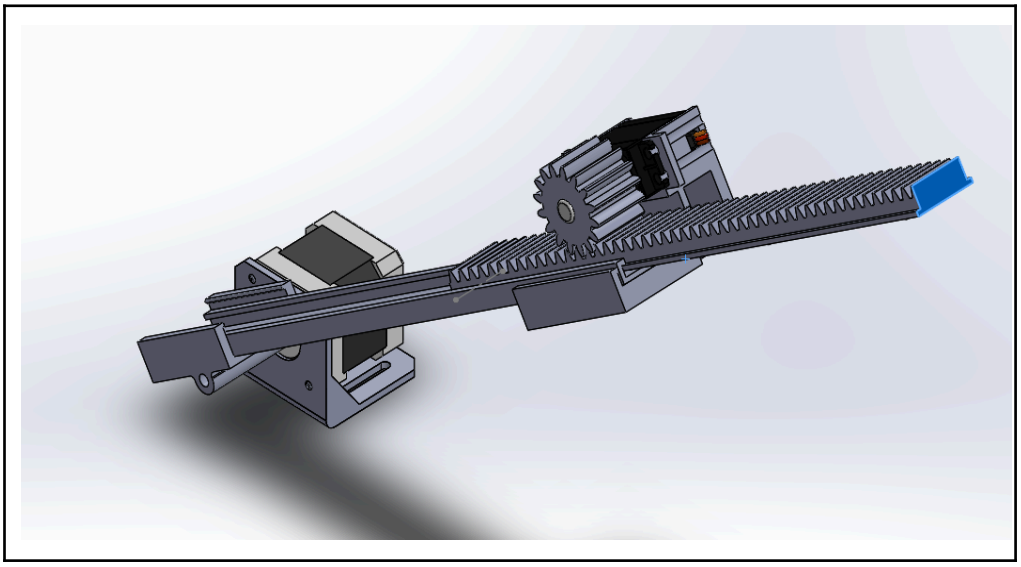


Figure 3.3: Close-Up of Rack and Pinion Mechanism

3. **3rd DOF: 360-Degree Rotation**
   A servo motor rotates the 4-bar mechanism and the attached camera platform, enabling a complete 360-degree view of the inspection target. This capability is particularly useful for inspecting RPV head adapters, as it allows the camera platform to rotate around the object, providing a comprehensive view from all angles without repositioning the robot itself.

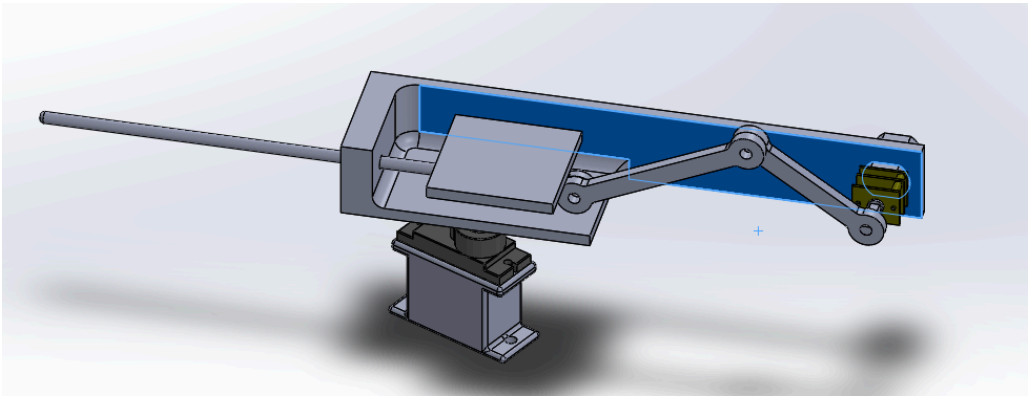


Figure 3.4: 360-Degree Rotation of the 4-Bar Mechanism

4. **4th DOF: 4-Bar Mechanism for Camera Adjustment**
   The 4-bar mechanism, powered by an N23 motor, converts rotational motion into translational motion, allowing precise camera extension and retraction (up to 75 mm). This enables fine adjustments to the camera's position, ensuring optimal image clarity without the need to reposition the robot

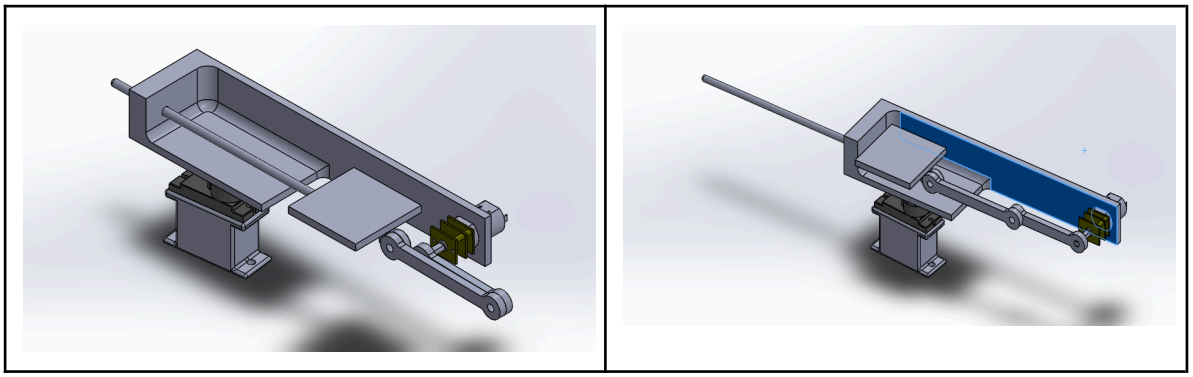


Figure 3.5: Retracting and Extending the 4-Bar Mechanism

The integration of these degrees of freedom ensures that the robot can adapt to various inspection scenarios, providing comprehensive coverage and improving inspection accuracy.

3.2. CAD Model of the Inspection Arena

To complement the robot's design, a specialized CAD model was developed for the inspection arena, ensuring a stable and safe environment for RPV head inspections. The arena incorporates key components such as the RPV head, support blocks, and radiation shields to optimize functionality.

3.2.1 Key Components of the Inspection Arena

1. **Reactor Pressure Vessel (RPV) Head**

   - The RPV head is a dome-shaped metallic structure, as shown in **Figure 3.6**.
   - Positioned at the center of the room, it is the primary focus of the inspection.
   - The head rests on four sturdy blocks that allow clearance underneath for the robot.

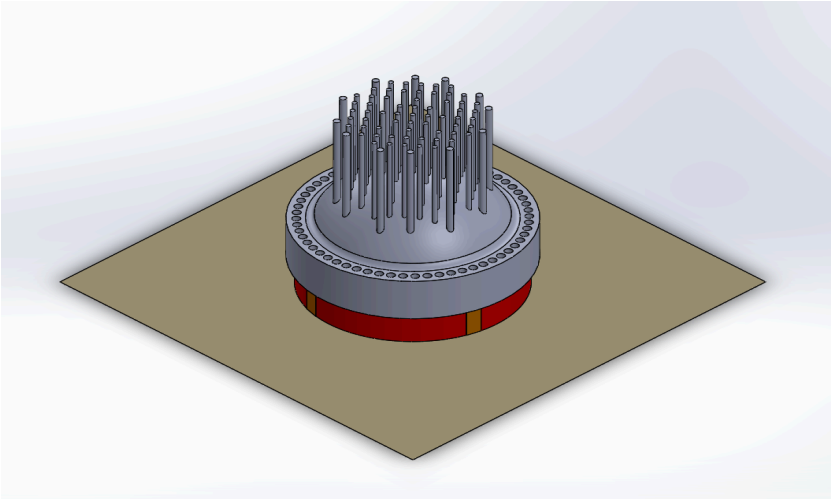


Figure 3.6: CAD Model of RPV Head on Support Blocks

2. **Support Blocks**

   - Four rectangular blocks support the RPV head, providing stability and maintaining its position.
   - These blocks are shown in **Figure 3.7**, designed to hold the curved surface securely.
   - Spacing between the blocks ensures enough clearance for the inspection robot to enter and operate beneath the head.

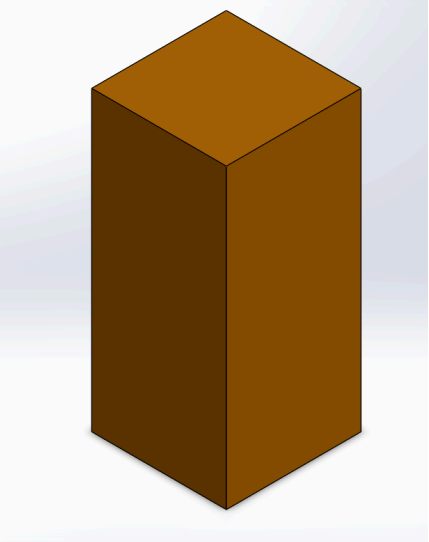


Figure 3.7: Close-Up of Support Block

3. **Radiation Shields**

    - To prevent radiation exposure, four curved shields are installed between the support blocks.
    - As shown in **Figure 3.8**, these shields are positioned to block radiation from escaping.
    - One shield can be temporarily removed to create an entry point for the robot while ensuring minimal radiation exposure.

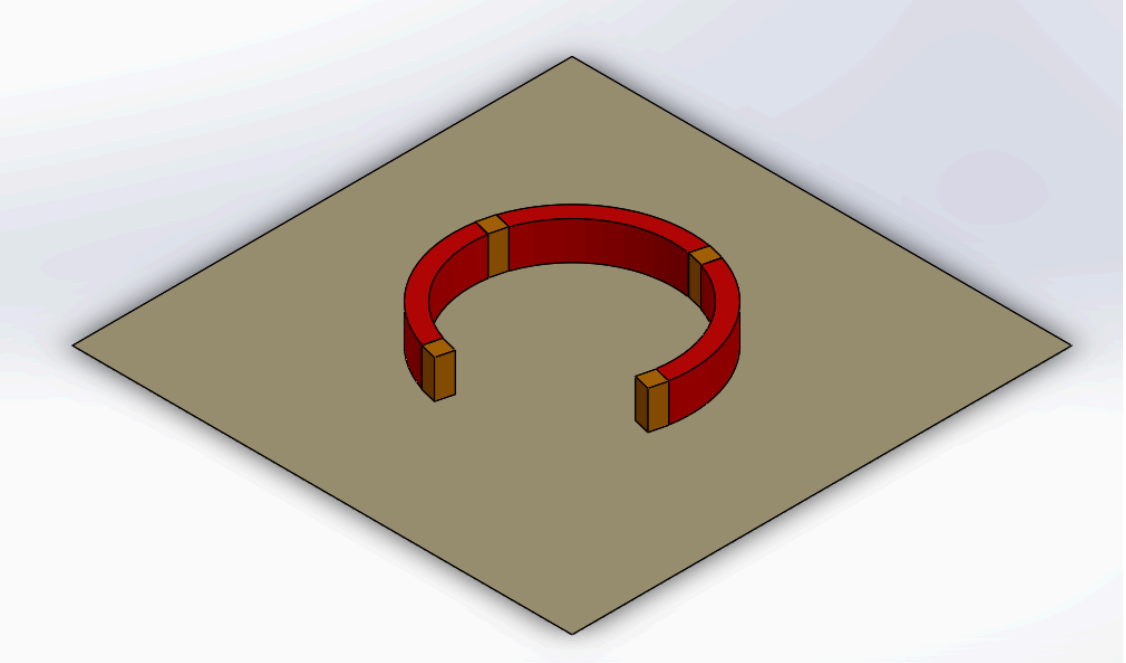


Figure 3.8: Radiation Shields with Robot Entry Point

3.2.2 Design Features of the Inspection Arena

- **Central Placement of RPV Head**
  The RPV head is centrally positioned to ensure equal accessibility from all sides. Its elevated placement on support blocks provides the robot with ample space for inspection beneath the head.

- **Radiation Safety Measures**
  Radiation shields are integral to operator and equipment safety. The removable shield design allows the robot to enter the inspection area without compromising protection.

- **Robot Accessibility**
  The arena's design includes a strategically positioned entry point and

sufficient clearance for the robot to navigate and perform detailed inspections in confined spaces.

3.3. Integration of Robot and Arena Design

The circular robot and the CAD-modeled inspection arena are designed to work in harmony. The robot's compact design, coupled with its advanced manipulator, allows it to navigate the arena's confined spaces and conduct thorough inspections. Meanwhile, the arena's structural and safety features ensure a controlled environment for efficient robot operations. Together, these systems provide a robust solution for RPV head inspections, enhancing safety, accuracy, and operational efficiency.

3.4. Conclusion

This chapter has presented the design and functionality of the circular inspection robot and its CAD-modeled inspection arena. The robot's 4-DOF manipulator and advanced mobility systems, combined with the arena's structural stability and radiation safety measures, demonstrate a comprehensive approach to RPV head inspections. By integrating these two components, the inspection process becomes safer, more efficient, and more effective, setting a new standard for inspections in nuclear power plant environments.

CHAPTER-4

PATH PLANNING AND SIMULATION USING MATLAB

4.1. INTRODUCTION

Efficient path planning is a critical component for autonomous robots, especially those operating in constrained environments such as the reactor pressure vessel (RPV) head. This chapter discusses the implementation of path planning and simulation for an autonomous inspection robot using MATLAB's Probabilistic Roadmap (PRM) function. The approach utilises a binary occupancy map representing the physical environment, followed by path planning and robot simulation using a circular robot model.

4.2. Binary Occupancy Map of the Arena

To simulate the robot's environment, a **binary occupancy map** was created. The arena is depicted on this map as a grid, with each cell designated as either:

- **Occupied (1):** Indicates an obstacle.
- **Free (0):** Represents an area the robot can traverse.

The binary occupancy map ensures that the robot can identify the safe paths and avoid collisions. The arena was modelled to reflect the real-world scenario the robot will encounter during its inspection tasks.

4.2.1 Occupancy Map Creation

The environment where the robot operates is represented as a **binary occupancy grid**, generated using MATLAB. To reduce computational cost, **all dimensions are scaled down** in the simulation while maintaining proportional accuracy to the physical environment. Key features include:

1. **Grid Parameters**:
    - The workspace is discretized into a grid, with a cell size of 0.1mm, representing the physical dimensions at a smaller scale.
    - The environment's boundaries are created as closed borders (occupied cells).

2. **Obstacle Representation**:
    - A cylindrical obstacle with a predefined radius is placed at the centre of the map. The cylinder includes an open sector between 0∘ and 90∘, simulating a doorway or gap for robot traversal.
3. **Robot Dimensions and Inflation**:
    - The robot's base is modelled as a circular shape with a diameter of 0.512 mm.
    - To account for the robot's size and ensure safe navigation, the map is "inflated" around obstacles, expanding occupied cells by a radius corresponding to the robot's clearance.
4. **Visualisation**:
    - Both the original and inflated occupancy maps are plotted for verification and saved for further use in the simulation.

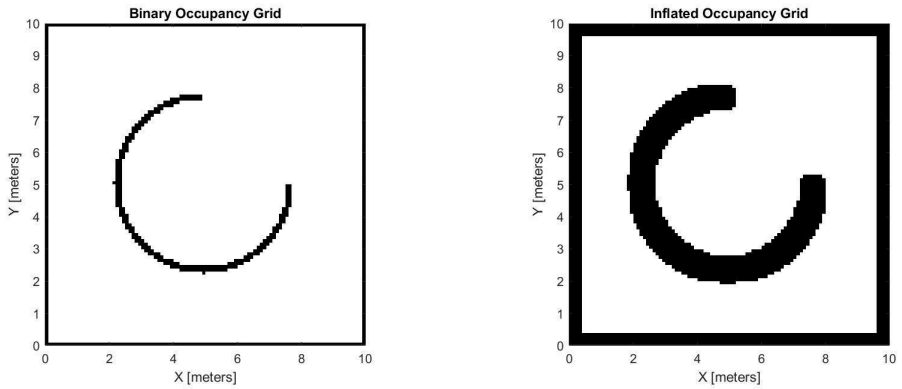


Figure 4.1. Binary and Inflated Occupancy Maps of the Robot Environment

4.2.2 Code Summary for Occupancy Map

```
% Create the binary occupancy map
occupancy_grid = zeros(grid_size_cells, grid_size_cells);
occupancy_grid(1:border_width_cells, :) = 1;
occupancy_grid(:, 1:border_width_cells) = 1;
for i = 1:grid_size_cells
    for j = 1:grid_size_cells
        distance = sqrt((i - center)^2 + (j - center)^2);
        angle = rad2deg(atan2(j - center, i - center));
        if (cylinder_outer_radius_cells - 1 <= distance && distance <=
cylinder_outer_radius_cells + 1) && ...
           ~(90 <= angle && angle <= 180)
            occupancy_grid(i, j) = 1;
        end
    end
end
% Inflate the grid to account for robot clearance
inflated_occupancy_grid = inflate_map(occupancy_grid,
```

```
robot_base_radius_cells);
% Save and visualize the maps
save('Final_occupancy_grid.mat', 'occupancy_grid');
save('Final_occupancy_grid_inflated.mat', 'inflated_occupancy_grid');
```

4.3. Path Planning Using PRM

The robot was instructed to follow a path created by the MATLAB Probabilistic Roadmap (PRM). PRM is a potent path-planning algorithm that joins randomly chosen places in the open environment to create a roadmap. From a user-specified start location to a goal point on the occupancy map, the robot utilises this roadmap to determine the shortest route.

1. **Binary Map Conversion**:
    - The previously created binary occupancy grid is loaded and converted into a binaryOccupancyMap object in MATLAB.
2. **User Input**:
    - Start and goal locations are selected interactively via keyboard inputs on the map.
3. **PRM Configuration**:
    - The roadmap is constructed with 100 nodes and a connection distance of 10 units.
    - The PRM finds an optimised path connecting the start and goal points while avoiding obstacles.
4. **Path Visualization**:
    - The roadmap and computed path are plotted for visualization.

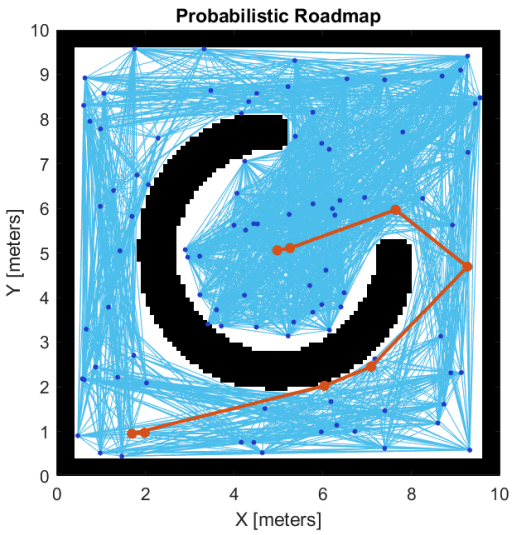


Figure 4.2. Optimized Path from Start to Goal Using PRM

4.3.1 Code Summary for Path Planning

```
map = binaryOccupancyMap(occupancy_grid, cell_size_mm); % Create BOM
% User selects start and goal points
[startLocationX, startLocationY] = ginput(1);
[goalLocationX, goalLocationY] = ginput(1);
prm = robotics.PRM(map);
prm.NumNodes = 100;
prm.ConnectionDistance = 10;
% Find and visualize the path
path = findpath(prm, startLocation, endLocation);
show(prm);
```

4.4. Robot Simulation

A **circular robot** is simulated to follow the planned path in the MATLAB environment. The simulation utilizes the **Pure Pursuit controller** to ensure the robot accurately tracks the waypoints of the path.

1. **Robot Initialization**:
    - The robot's initial pose, including position and orientation, is set based on the start location.
    - When the robot arrives at the desired position, a goal radius is established to end the simulation.
2. **Dynamic Updates**:
    - The robot's position and orientation are continuously updated based on the controller's velocity and angular velocity outputs.
    - The distance to the goal is recomputed at each step to monitor progress.
3. **Visualization**:
    - The robot's path is animated in real time on the occupancy map, showcasing its trajectory as it navigates towards the goal.

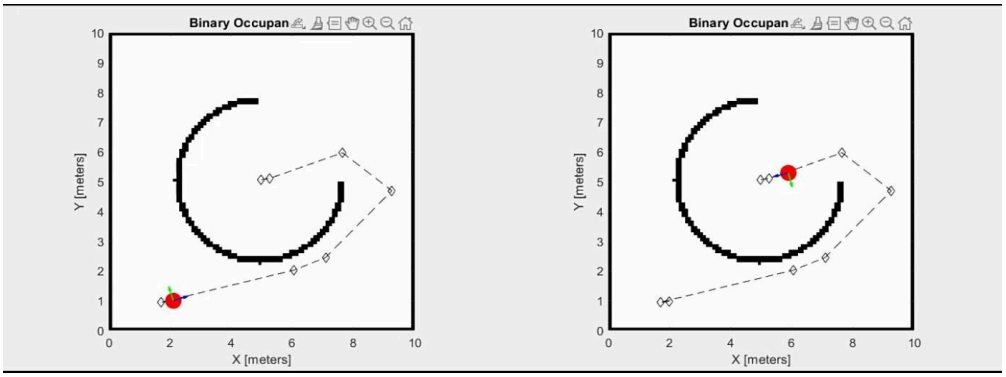


Figure 4.3: Robot at Start and Goal Positions in the Path Planning Simulation

4.4.1 Code Summary for Robot Simulation

```
% Initialize robot state and controller
controller = robotics.PurePursuit('Waypoints', path);
robotCurrentPose = [path(1,:) 0]'; % Start location and orientation
distanceToGoal = norm(path(1,:) - path(end,:));
% Simulate robot motion
while distanceToGoal > goalRadius
    % Compute control inputs
    [v, omega] = controller(robotCurrentPose);
    % Update robot pose
    theta = robotCurrentPose(3);
    robotCurrentPose = robotCurrentPose + [v * cos(theta) * sampleTime;
                                           v * sin(theta) * sampleTime;
                                           omega * sampleTime];
    % Recompute distance to goal
    distanceToGoal = norm(robotCurrentPose(1:2) - path(end,:));
end
```

4.5. Results and Discussion

- **Map Visualization**:
  - The generated occupancy maps clearly depict the environment, including obstacles and safe navigation areas.
- **Path Planning**:
  - The PRM successfully computes feasible paths avoiding obstacles.
  - The paths are optimised based on the number of nodes and connection distances.
- **Simulation**:
  - The robot smoothly follows the generated path, demonstrating the effectiveness of the Pure Pursuit controller.
  - Visualisation provides a comprehensive understanding of the robot's movement and performance in the planned environment.

4.6. Conclusion

This chapter highlights the integration of path planning and robot simulation in MATLAB. By leveraging tools like the PRM and Pure Pursuit controller, an effective navigation solution was demonstrated for the autonomous inspection robot. The approach ensures safe and efficient traversal of the RPV head environment.

CHAPTER-5

Human-Machine Interface for Robot Control

5.1. INTRODUCTION

The Human-Machine Interface (HMI) plays a crucial role in facilitating the interaction between the operator and the inspection robot. Developed using the Windows Presentation Foundation (WPF) framework, the HMI provides a secure and user-friendly platform for controlling the robot and managing inspection tasks. The interface comprises several pages, including **Login**, **Home**, **Manual Control**, **Simulation Control**, and **Autonomous Operation**, each designed to provide specific functionalities for the robot's operation and monitoring.

With real-time feedback and an intuitive layout, the HMI enhances the operator's ability to control, configure, and execute tasks efficiently. It also supports image and video inspection, offering a complete solution for handling various inspection scenarios.

The HMI is structured around four core pages:

- **Home**
- **Manual Control**
- **Simulation Control**
- **Autonomous Operation**

Each of these pages serves a specific purpose, from basic connection settings to manual and automated robot control. The interface seamlessly integrates with the robot's operational capabilities, allowing the user to interact with and monitor the robot effectively. This structure enhances workflow during critical inspection tasks, ensuring precise control and monitoring capabilities at every stage.

5.2. Objective

The primary objective of the HMI is to develop a simple, visually intuitive interface that provides secure, efficient control over the inspection robot. Specifically, the interface aims to:

- Enable easy navigation between various control options.
- Provide real-time feedback on the robot's operations.

- Allow users to configure, manage, and execute inspection tasks in both manual and autonomous modes.
- Facilitate seamless communication between the robot and the HMI for responsive task execution.

By achieving these objectives, the HMI enhances the operator's ability to manage complex inspection operations with minimal effort, while ensuring robust security and system reliability.

5.3 Login Page

The **Login Page** ensures secure access to the robot's control system. The page includes:

- **Username and Password Fields:** Users must enter valid credentials to access the system.
- **Login Button:** Once the credentials are entered, clicking this button authenticates the user. If the login is successful, the user is taken to the Home Page; otherwise, an error message is displayed.

This page is crucial for ensuring that only authorized personnel can access the robot's controls, protecting the system from unauthorized usage.

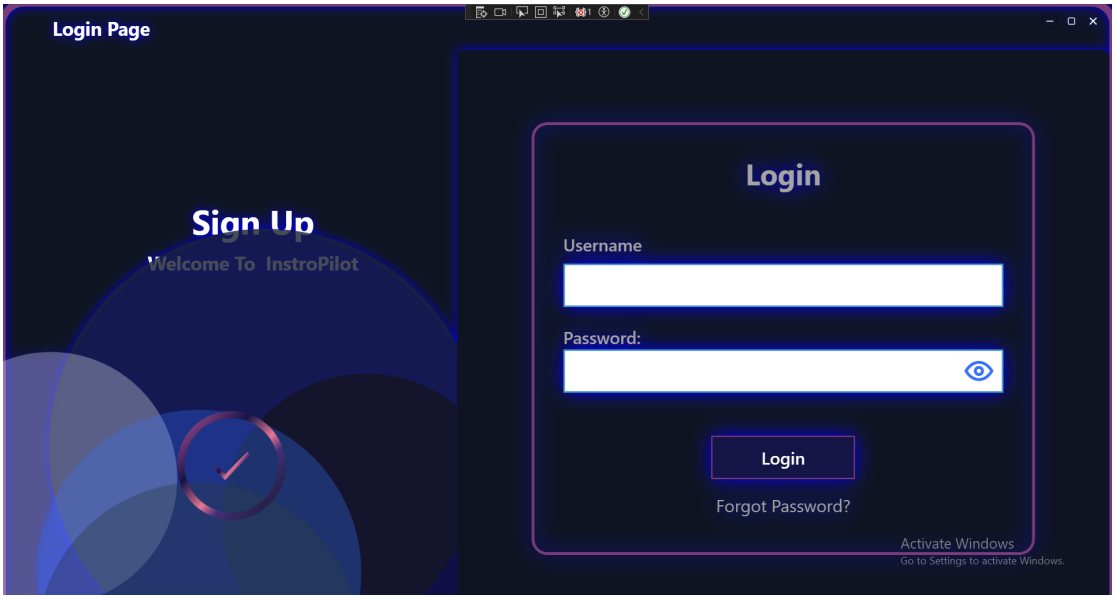


Figure 5.1: Login Page

5.3.1 User Interface Design

The design of the HMI prioritizes clarity and ease of use. The layout is structured around a navigation bar located on the left side of the interface, which allows users to

quickly switch between different control pages. On the right side, dynamic content relevant to each selected page is displayed, ensuring the user has all the necessary controls and information at hand.

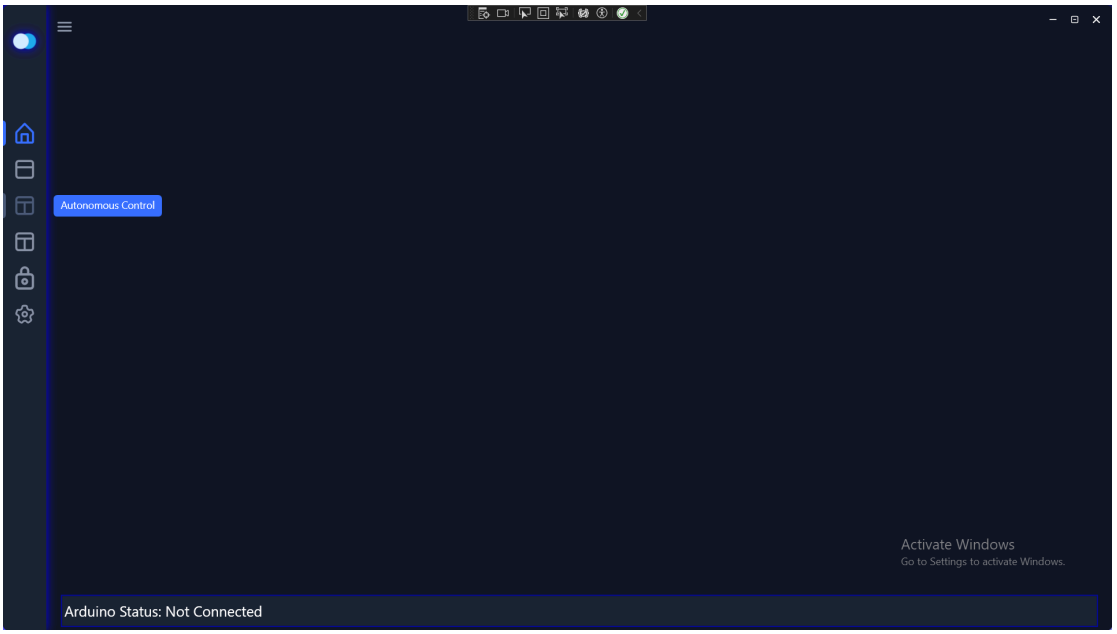


Figure 5.2: Navigation Bar

5.3.2 Home Page

The **Home Page** is the central hub for managing the robot's tasks and inspection activities. In addition to connecting to the robot, it offers several key features that support both task execution and image/video inspection:

- **Connection Area:** A dedicated connection area facilitates the interaction between the user and the inspection robot. User can connect the robot to an Arduino using the Serial Monitor. This area includes:
    - **Editable Combo Box:** Allows the user to select the appropriate baud rate and port number for communication.
    - **Connect Button:** Once the correct settings are selected, pressing the **Connect** button initiates the connection process with the Arduino. The connection status is dynamically updated on a status bar, and the button changes to **Disconnect** once the connection is established. Pressing the **Disconnect** button terminates the connection and updates the status bar accordingly.
- **Task Loading and Execution:** Users can select a CSV file containing pre-configured tasks for the robot. Upon selection, the tasks are automatically loaded into a **DataGrid**, where they are displayed sequentially along with their status (either **executed** or **pending**). The operator can then send these tasks to the robot for **autonomous execution**. As each task is processed, the

interface dynamically updates the status of each task in real time, allowing the operator to monitor the progress of the autonomous operation.

- **Image Inspection:** The Home Page also allows users to select a folder containing inspection images. The user can navigate through these images one by one, using **Next** and **Previous** buttons to inspect each image individually. A **Detect** button applies the robot's detection algorithm to the current image, and a **Save** button saves the inspected image. Users can set a threshold value via a **ComboBox**, adjusting the detection sensitivity.
    - **Auto-Inspection:** By pressing the **Auto** button, users can automate the inspection of all images in the selected folder. The system will load each image, inspect it according to the threshold setting, and save the results in a new folder created on the desktop. This process streamlines batch inspection, making it faster and more efficient.

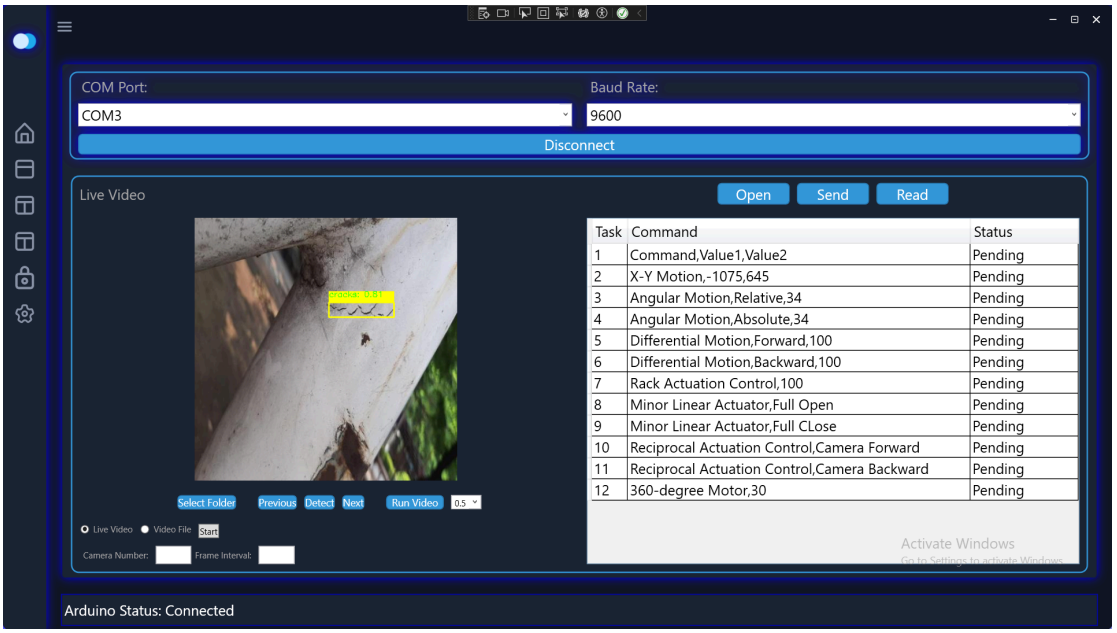


Figure 5.3: Image Detection on Home Page

- **Video Inspection:** The Home Page also supports video inspection for both pre-recorded and live video streams:
    - **Pre-recorded Video:** Users can select the **File** radio button and browse for a pre-recorded video file. After selecting the video and setting a frame interval to reduce computational cost (i.e., detecting only after a specified number of frames), the system inspects the video and saves the processed output in a folder named **VideoDetection** on the desktop, with the filename **Output_Filename.mp4**.
    - **Live Video:** Alternatively, users can select the **Live** radio button to connect to a live camera feed. The operator enters the camera number (e.g., 0, 1, 2) and the frame interval. When the **Start** button is pressed,

the system inspects the live video and saves the processed feed in the **VideoDetection** folder on the desktop.

The Home Page, therefore, serves as a multifunctional interface for both task execution and image/video inspection, significantly enhancing the robot's usability and operational flexibility.

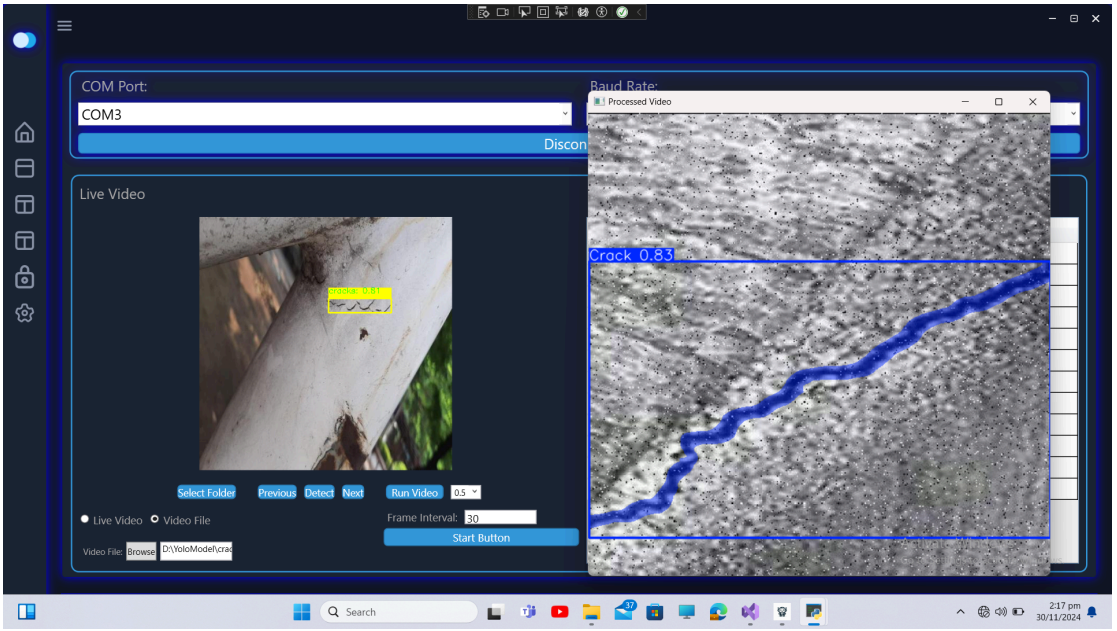


Figure 5.4: Live Video Detection on Home Page

5.3.3 Manual Control Page

The **Manual Control Page** is where users have direct control over the robot's movement and mechanisms. This page is divided into several sections (referred to as "borders"), each focusing on a specific aspect of the robot's operation:

- **Chassis Control:** This section contains buttons to control the movement of the robot's chassis. Buttons for **Forward**, **Backward**, **Left**, **Right**, and **Stop** send respective commands to the robot's control system (Arduino), initiating the movement of the robot in the desired direction.
- **Minor Actuator Control:** This section includes buttons labeled **Manipulator Up** and **Manipulator Down**, which control the minor actuator. Pressing these buttons sends signals to adjust the actuator's position to up or down as needed.
- **Rack Actuation Control:** Similar to the minor actuator controls, this section manages the rack actuation, providing **Full Open** and **Full Close** options. It also includes a **Rack Stop** button to halt the rack at the desired position.
- **Camera Control:** The Camera Control section allows the operator to adjust the orientation of the robot's camera. Buttons for **Clockwise** and

Counterclockwise rotation send commands to the robot's camera system to adjust its viewing angle.

- **Reciprocal Actuation Control:** This section controls the four-bar mechanism, allowing users to move the camera platform to either the fully open or fully closed position with dedicated buttons.

Each section sends commands to the Arduino, which translates these inputs into precise actions, providing the user with fine control over the robot's mechanical functions.

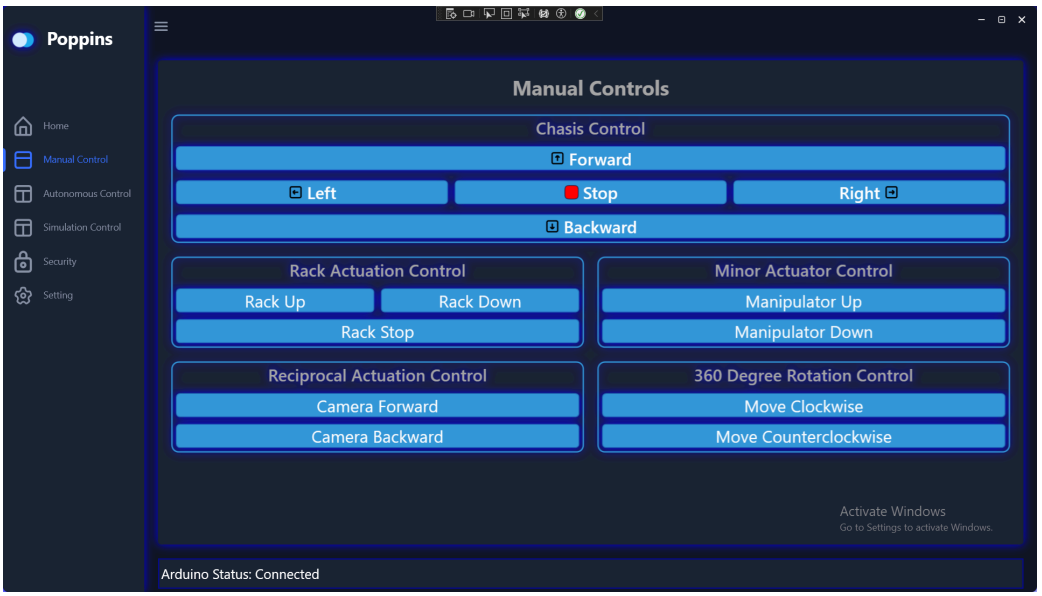


Figure 5.5: Manual Page Control

5.3.4 Simulation Page

The **Simulation Page** allows users to test and visualize the robot's path planning in a virtual environment. This page provides an interactive interface where users can input start and goal points, execute the path planning algorithm, and observe the simulated movement of the robot along the calculated path.

- **Key Features:**
  - **User Input Section:**
    - **Origin Point:** A combo box where users can select or enter the starting coordinates (X, Y) for the desired location.
    - **Goal Point:** A combo box for selecting or entering the destination coordinates (X, Y).
  - **Simulation Execution:**
    - A **Start Simulation** button that triggers the MATLAB path planning script.

        - Clicking the button sends the entered coordinates to MATLAB, which calculates the optimal path and displays the simulation window.
    - **Simulation Display:**

      After the path planning execution, a new window opens showing the robot navigating the calculated path from the start to the goal point in real time.
- **Workflow:**
    - **Enter Coordinates:**
        - The user enters the origin and goal points in the respective text boxes on the Simulation Page.
    - **Start Simulation:**
        - Clicking the **Start Simulation** button sends these coordinates to the MATLAB file, which processes the path planning algorithm to calculate the optimal path.
    - **View Simulation:**
        - The MATLAB simulation window opens, displaying the robot following the generated path in real time, allowing the user to visually verify the path planning.
- **Purpose:**

The **Simulation Page** offers a practical way to verify the path planning algorithm before deploying commands to the actual robot. This allows the operator to simulate and observe the robot's movement, ensuring smoother and more efficient performance during real-world operations. By testing the path in a virtual environment first, users can identify any potential issues with the path planning before actual execution, improving reliability and performance.

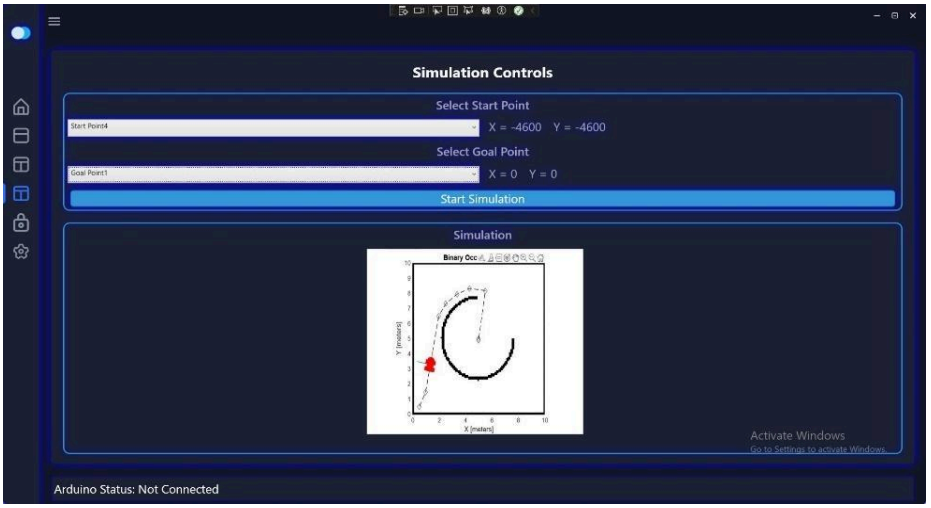


Figure 5.6: Simulation Page Control

5.3.5 Autonomous Operation Page

The **Autonomous Operation Page** allows users to configure and manage the robot's automated tasks. This page is integral to making the robot self-sufficient during inspections, providing options for setting up and executing inspection routines without manual intervention. Key features of this page include:

- **Task Configuration:** Users can create, open, and save task configurations in CSV files. This ensures that tasks can be prepared in advance and executed consistently during inspections.
- **Motion and Checking Commands:** Operators can configure the robot's movements and sensor checks by inputting the desired parameters. This setup allows the robot to follow a predefined inspection path, adjust its sensors, and check for specific conditions autonomously.

The Autonomous Operation Page is designed to streamline the inspection process by reducing the need for constant human oversight, thereby improving task efficiency and reducing operational delays.

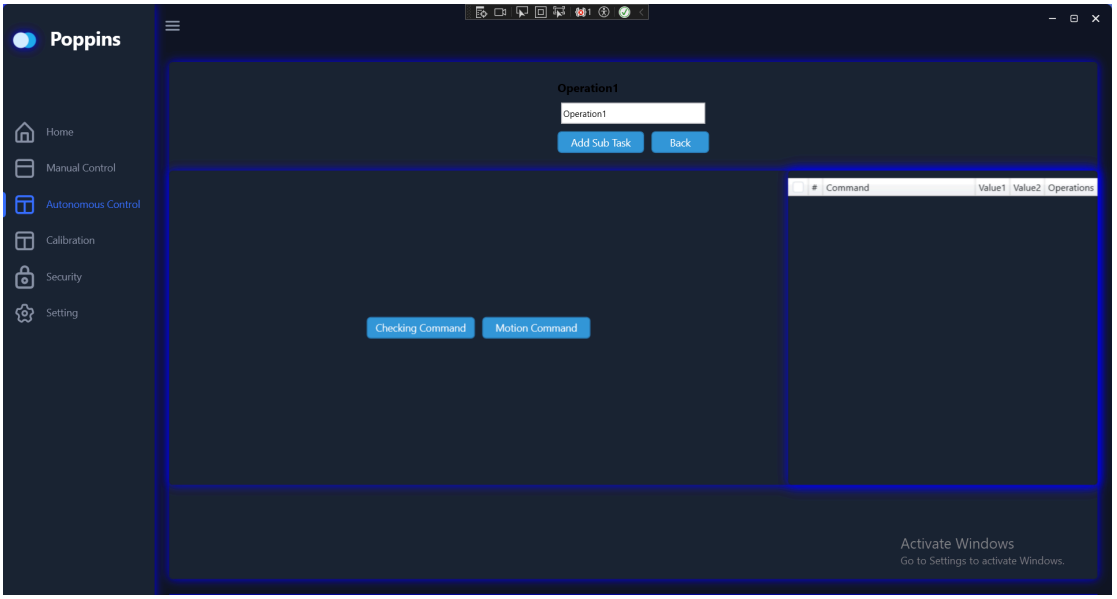


Figure 5.7: Autonomous Page Control

5.3.6 Automation Page Features

The **Automation Page** is central to managing and controlling the robot's inspection tasks. It offers a range of features designed to simplify task setup and execution:

- **File Management:** Users can create new task configurations or open existing ones from CSV files. This allows the robot to execute tasks based on predefined instructions.

- **Task Management:** The page provides tools for adding, editing, deleting, and updating task configurations. These tools ensure that the robot can adapt to changing conditions or new requirements during the inspection.
- **Real-Time Feedback and Monitoring:** The interface provides real-time feedback, allowing the user to monitor the progress of tasks and make adjustments as needed. This feature is essential for ensuring that tasks are executed accurately and efficiently.
- **Motion and Actuator Control:** The Automation Page allows users to configure various types of motion commands, such as chassis movement, actuator positions, and camera adjustments. These controls give the robot the flexibility to perform complex inspection tasks autonomously.

This page plays a crucial role in enhancing the robot's performance, enabling the operator to manage tasks efficiently and respond to real-time feedback for optimal results.

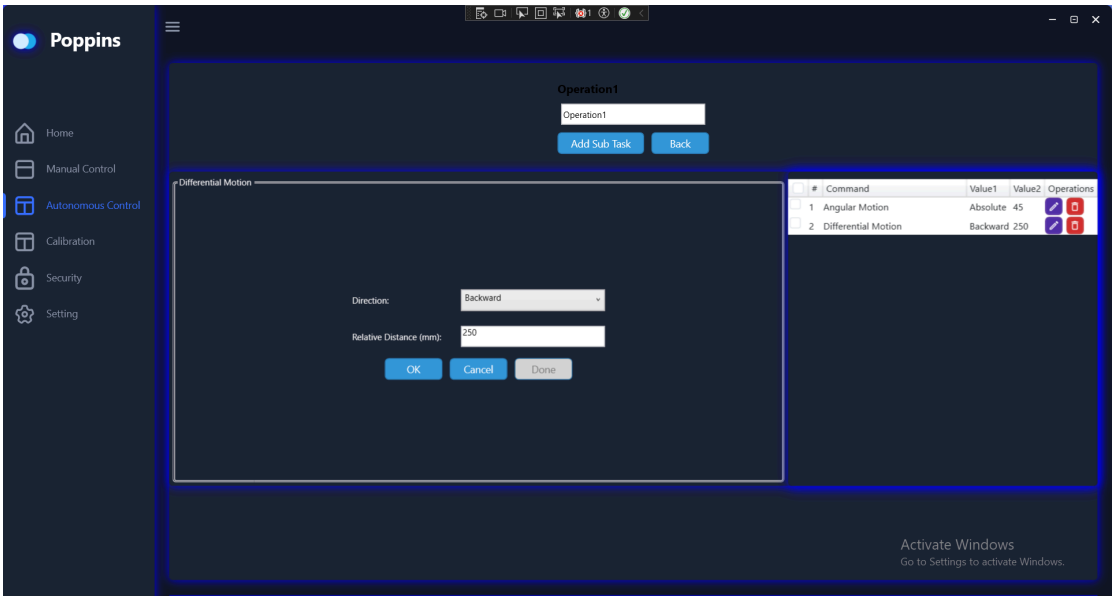


Figure 5.8: Autonomous Page Task management

5.4. Real-Time Interaction and Monitoring

One of the key advantages of this HMI is its ability to provide real-time interaction between the operator and the robot. Whether in manual or autonomous mode, the system constantly updates the user on the robot's status, ensuring that any issues can be addressed immediately. This feedback loop is vital for:

- **Dynamic Control:** The user can adjust the robot's movements, sensor inputs, or task parameters on the fly.

- **Status Monitoring:** Real-time status updates allow the operator to track the progress of inspection tasks, ensuring they are completed as expected.
- **Image/Video Inspection:** Operators can inspect images and videos in real time, applying detection algorithms and saving results dynamically.
- **Error Handling:** Immediate feedback enables the operator to intervene if any issues arise during the robot's operation, reducing downtime and improving task completion rates.

By integrating real-time interaction, the HMI significantly enhances the robot's operational effectiveness during critical inspection tasks.

5.5. Conclusion

The HMI developed for this inspection robot successfully meets its objectives of user-friendliness, security, and operational efficiency. Using the WPF framework, the interface is easy to navigate, visually intuitive, and provides comprehensive control over the robot's various functionalities. The **Home**, **Manual Control**, **Simulation**, and **Autonomous Operation** pages offer clear and structured access to the robot's core operations, allowing the operator to manage both manual and automated tasks seamlessly.

CHAPTER-6

METAL CRACK DETECTION USING DEEP LEARNING TECHNIQUES

6.1. INTRODUCTION

Metal cracks are critical flaws in industrial systems that compromise safety and operational efficiency. This chapter outlines the development of an autonomous system for detecting cracks in metal surfaces using deep learning models, specifically focusing on segmentation. The process involves dataset creation, data augmentation, annotation, and model training using **YOLO v11-seg**. A comparative analysis of the trained models' performance follows.

6.2. Dataset Preparation

6.2.1 Data Collection

The dataset consists of **1,111 images for training** and **246 images for validation**. These were sourced from various industrial applications and repositories, with a focus on metal cracks.

6.2.2 Data Cleaning

Images were filtered for relevance and quality. Low-resolution or irrelevant images were excluded to ensure high-quality input for training. This process helped standardize the dataset and improve model performance.

6.2.3 Data Augmentation

To enhance the dataset's generalization, several augmentation techniques were applied using the **Albumentations** library, including:

- *Blurring*, *Conversion to Grayscale*, *Flipping*, and *Rotating, Random scaling and translation*.



Figure 6.1: Data Augmentation Visualization

6.2.4 Data Annotation

Using **Computer Vision Annotation Tool (CVAT)**, images were annotated with bounding boxes and segmentation masks to label crack locations accurately. The annotation focused on varying crack sizes and shapes to cover the diversity of real-world metal surface flaws.



Figure 6.2: Data Annotation Visualization

6.2.5 Data Splitting

The data was split into training (1,111 images) and validation (246 images) sets, ensuring that the validation set was representative of the data in the training set for effective model evaluation.

6.3. Model Training

6.3.1 Model Overview

The YOLO v11-seg model was chosen for its superior ability to perform fast and accurate object detection and segmentation. This model's segmentation head was specifically tailored for detecting cracks on metal surfaces, making it well-suited for the task at hand. The architecture's efficiency allows for precise crack segmentation, enabling real-time performance while maintaining high accuracy. The training was optimized with a carefully selected set of hyperparameters, ensuring robust model performance across varying conditions. With a focus on accuracy and speed, YOLO v11-seg proved to be an ideal choice for this industrial crack detection application.

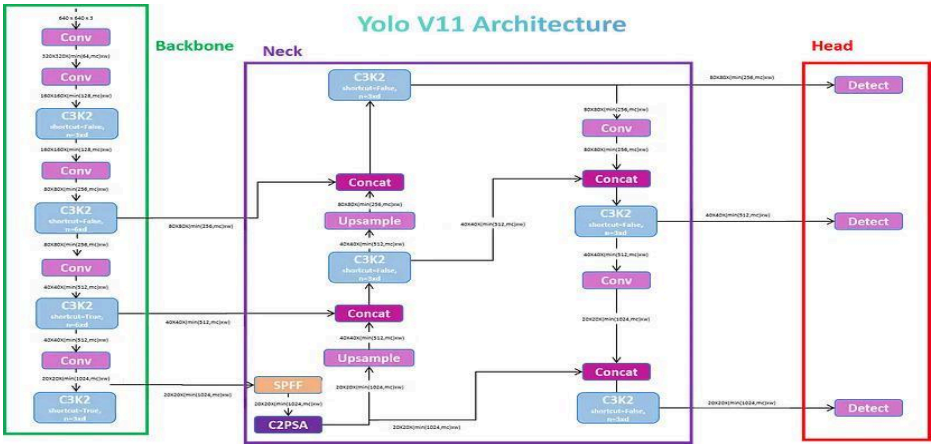


Figure 6.3: YOLOv11-seg Model Architecture

Key Model Parameters:

- **Epochs**: 160
- **Batch Size**: 16
- **Image Size**: 640x640
- **Optimizer**: AdamW
- **Learning Rate (initial)**: 0.002
- **Momentum**: 0.9
- **Automatic Mixed Precision (AMP)**: Enabled for faster computation.

6.3.2 Training Pipeline

The training process was executed on a Tesla T4 GPU using the following steps:

1. **Optimizer Selection**: An AdamW optimizer was automatically configured for optimal performance, ignoring manual settings like lr0=0.01 and momentum=0.937.
2. **Augmentation**: Real-time augmentations, such as random flips and contrast adjustments, were applied during training to improve the model's robustness.
3. **Loss Function**: The loss function was a combination of object detection loss and segmentation mask loss to ensure accurate crack localization and segmentation.
4. **Training Environment**: TensorBoard was used to monitor training metrics, and the entire process was logged for further analysis.

6.4. Model Evaluation

6.4.1 Metrics

The models were evaluated based on the following performance indicators:

- **Precision**: Accuracy in detecting true positives (cracks).
- **Recall**: Ability to detect all relevant cracks.
- **Mean Average Precision (mAP)**: Evaluated at multiple IoU thresholds (mAP50 and mAP50-95).
- **Processing Speed**: Time taken for preprocessing, inference, and postprocessing.

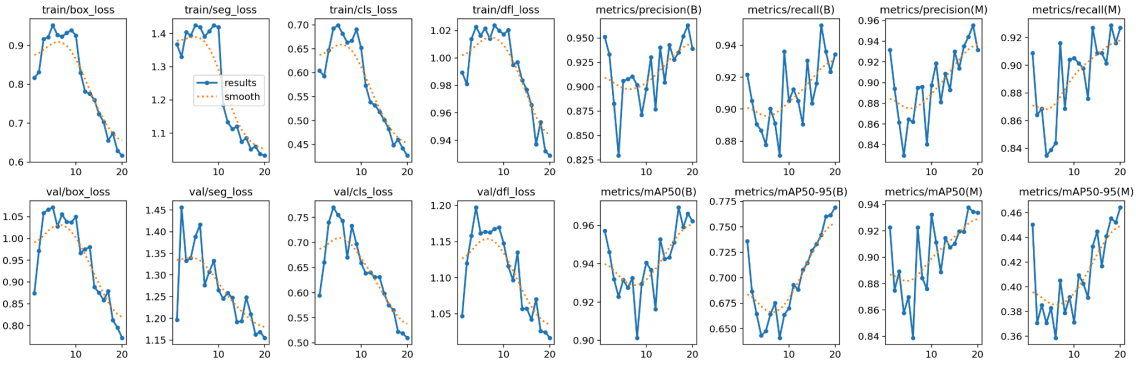


Figure 6.4: Model Results and Evaluation Metrics

6.4.2 Performance Comparison

The image below summarizes the performance of the models trained under different configurations. The top-performing model was Model 4 (Train6), which achieved the highest overall fitness score

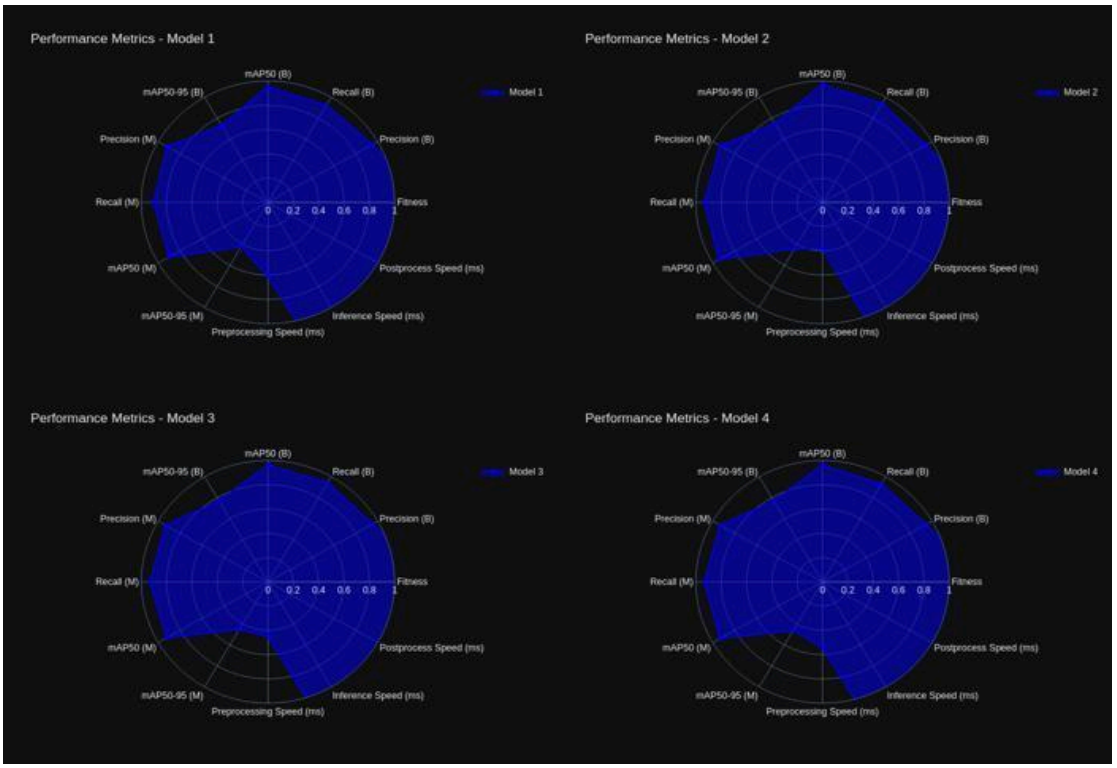


Figure 6.5: Models Evaluation And Comparison Results

6.4.3 Key Insights

- **Model 4 (Train6)** showed the highest fitness score, with well-balanced metrics across precision, recall, and inference speed.
- **Model 2 (Train4)** performed fastest in terms of inference and processing speed, making it ideal for real-time applications.
- Other models demonstrated competitive performance but were slightly behind in specific metrics.

6.4.3 Results

Model 4 (Train6) delivered the best performance in crack detection and segmentation, with precision, recall, and mAP scores consistently above 0.90. The model achieved high accuracy while maintaining efficient processing speeds, making it suitable for real-time applications. The results and confusion matrix, shown below, confirm the model's robustness, with minimal false positives and negatives. These findings

validate Model 4 as the most reliable choice for crack detection in industrial environments.

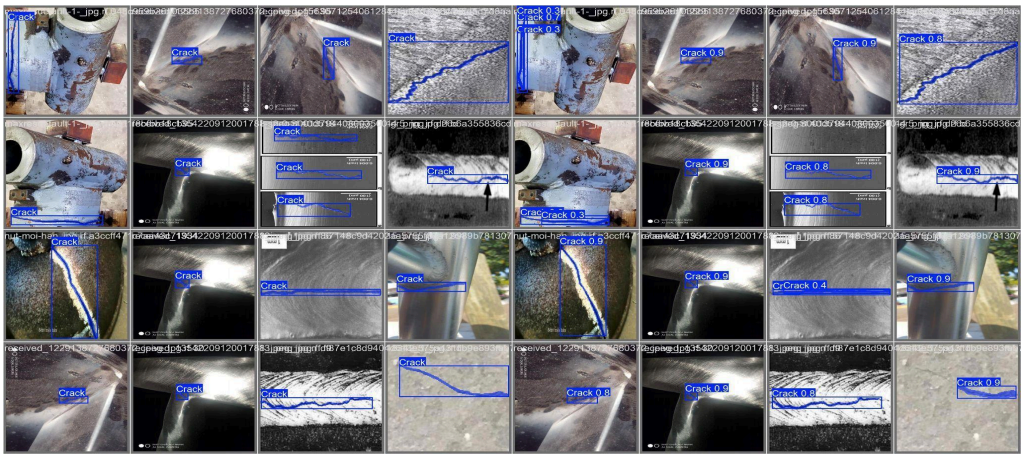


Figure 6.6: Model Predictions

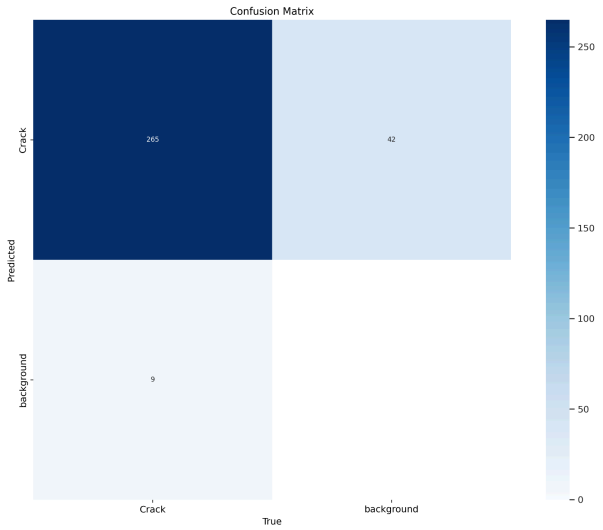


Figure 6.7: Confusion Matrix

6.5. Conclusion

This chapter outlined a deep learning-based approach to metal crack detection, leveraging YOLO v11-seg for crack segmentation. Model 4 was chosen for its exceptional balance of accuracy and robustness, demonstrating superior performance across key metrics, including fitness, precision, recall, and mAP scores. Its ability to effectively detect and segment cracks with high accuracy made it the ideal choice for deployment in industrial applications. Additionally, Model 2 showcased excellent speed, making it well-suited for real-time detection tasks. Together, these models highlight the potential of deep learning in ensuring safety and efficiency in industrial environments.

Chapter 7

SYSTEM INTEGRATION WITH HARDWARE AND HMI

7.1 INTRODUCTION

This chapter outlines the process of integrating various components to develop a fully functional Human-Machine Interface (HMI) for the inspection robot. The HMI serves as the central control and visualization platform, facilitating interactions between hardware, deep learning models, and external computational tools. By combining technologies like WPF, Python, MATLAB, and ONNX, the system provides a seamless and user-friendly experience.

7.2 Hardware Integration

To establish communication between the HMI and the robot's hardware, a serial interface is used. The HMI connects to an Arduino microcontroller, enabling real-time command transmission and data reception.

7.2.1 Logic Overview:

1. **Connection Management:** The HMI checks for active connections and ensures a clean setup before establishing a new one.
2. **Command Transmission:** Commands for controlling the robot are sent via a serial interface.
3. **Data Reception:** The HMI receives and processes data from the Arduino, like sensor readings or status updates.

This hardware integration allows the user to control the robot directly through the HMI.

7.3 Integration of Deep Learning Model

7.3.1 ONNX

The Open Neural Network Exchange (ONNX) is a standard format that helps share machine learning models between different tools. For example, you can train a model in one tool, like PyTorch, and use it in another, like TensorFlow, without making many changes. This makes it easier to work with different tools and use the strengths of each. ONNX saves time and effort by making machine learning models more flexible and ready to use across various platforms.

A YOLO-based deep learning model was trained to detect different types of cracks, as explained in **Chapter 6**. To make it work smoothly in the Windows Presentation Foundation (WPF) application, the model was converted to the **ONNX format**. Using ONNX made the integration easier and more flexible, allowing the powerful crack detection features of YOLO to be used effectively in the system. This approach simplified deployment and ensured the model could meet the application's needs efficiently.

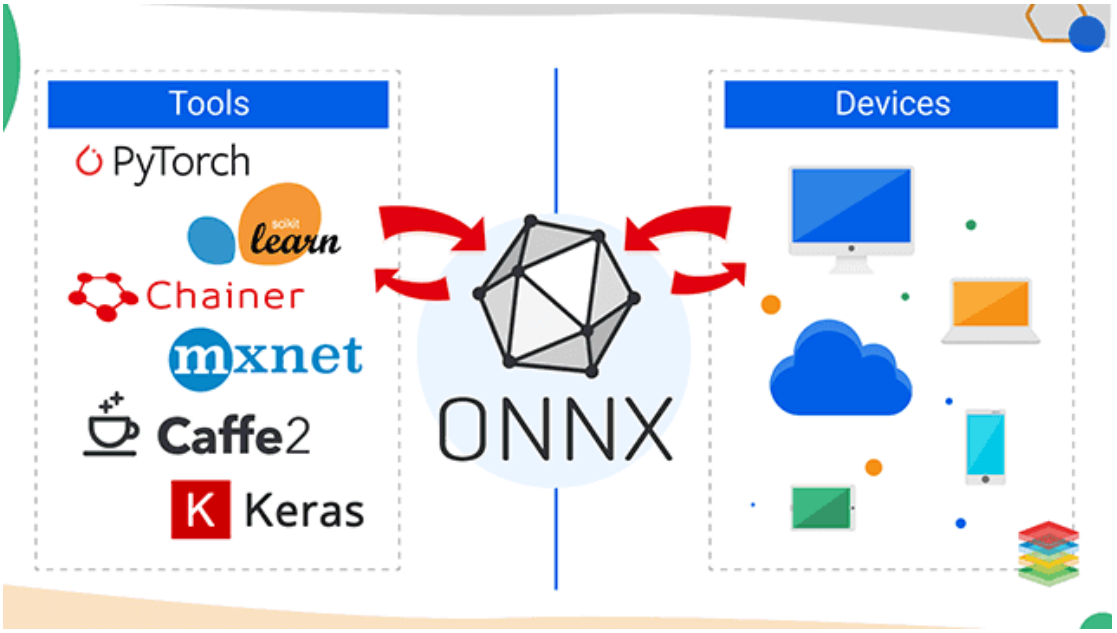


Figure 7.1: ONNX Format Conversion

7.3.2 Logic Overview:

1. **Model Loading:** The ONNX model is integrated into the HMI using a high-performance runtime library.
2. **Inference Execution:** Input data, such as images or video frames, is processed by the model to detect cracks in real-time.
3. **Output Display:** The detected cracks are highlighted and visualized on the HMI, providing users with immediate insights into the inspection process.

The deep learning model enhances the robot's functionality by automating the defect detection process, improving accuracy and efficiency.

7.4 Live Video Processing with Python

To support live video analysis, the HMI is integrated with Python scripts. This enables real-time object detection and analysis during robot operation.

7.4.1 Logic Overview:

1. **Script Execution:** The HMI triggers a Python script, passing parameters such as video source, detection interval, and output location.
2. **Video Processing:** The script performs live detection using the YOLO model and saves the processed video with highlighted detections.
3. **Result Handling:** The output video and any additional results are made accessible through the HMI for further analysis or review.

This integration allows the system to leverage Python's flexibility for high-performance video processing while maintaining a seamless user experience.

7.5 MATLAB Integration

The HMI also incorporates MATLAB for advanced computations, such as path planning. This is particularly useful for scenarios requiring precise navigation or algorithmic optimization. The complete path planning process, including algorithms and methodologies, is discussed in detail in **Chapter 4**.

7.5.1 Logic Overview:

1. **Data Input:** Users provide parameters, such as starting and goal coordinates, through the HMI.
2. **Script Execution:** The HMI sends this data to MATLAB, where PRM algorithms calculate the optimal path or perform other computational tasks
3. **Output Retrieval:** The results are sent back to the HMI, enabling users to visualize the computed path or use the data for further actions.

This integration bridges the gap between computational power and user interaction, ensuring sophisticated processing capabilities are easily accessible.

7.6 System Flow and Interaction

The complete system integrates hardware, deep learning, video processing, and computational tools into a unified workflow:

1. **HMI as the Central Hub:** Users interact with the robot via the HMI, which handles all inputs and outputs.
2. **Seamless Communication:** The HMI manages data flow between the Arduino, Python, and MATLAB environments.

3. **Real-Time Processing:** Deep learning and video analysis operate concurrently with hardware commands, ensuring responsiveness.
4. **User-Friendly Feedback:** Processed data and results, such as detected defects or computed paths, are presented clearly on the HMI.

7.7 Conclusion

The integration of these diverse technologies demonstrates the flexibility and robustness of the system. By combining hardware control, real-time video processing, deep learning, and advanced computations, the HMI transforms the inspection robot into a versatile tool for defect detection and navigation.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 CONCLUSION

This project focused on developing a circular robot with an integrated manipulator for inspecting Reactor Pressure Vessels (RPVs). The objectives included designing the CAD model for the robot and its arena, implementing path planning and simulation, creating an HMI to control the robot's hardware, training a deep learning model for crack detection, and integrating these systems. The system successfully connects the HMI with MATLAB for path planning and 2D simulation, deploys the trained model for crack detection in both live and recorded video, and communicates with the hardware for robot control.

8.2 Future Work

While we achieved significant milestones, there are areas where future work can enhance the system further:

1. **Integration of HMI and Deep Learning Model on Microcontroller**:
   We have created an executable file for the HMI and converted the trained model into TensorFlow Lite (TFLite) format. This allows future users to directly integrate the HMI and model into a Raspberry Pi or other microcontrollers, enabling real-time crack detection on a compact system.

2. **Advanced Simulations Before Hardware Operations**:
   To ensure safety in hazardous environments, we developed a simulation system using MATLAB Simscape Multibody. Commands can be stored in a file line by line and sent to the robot via the HMI. This allows users to test operations in the simulation before executing them on the real robot, ensuring accuracy and safety. However, the robot's chassis does not yet fully follow the generated path. Future work can focus on improving chassis movement control and its integration with the HMI.

3. **Standalone HMI Application with MATLAB Integration**:
   Currently, the HMI requires MATLAB to be installed on the user’s computer for simulation. Future work can involve making the application fully standalone by embedding MATLAB functionalities directly into the HMI. Although we identified the steps to achieve this, it required MATLAB's

premium features, which we could not access. These steps and all resources are available in the repository.

4. **Repository and Resources**:
   All the resources, including the TFLite model, HMI files, simulation files, and instructions for further development, are provided in the project repository. This ensures future developers can easily continue the work without starting from scratch.

This project provides a strong foundation for robotic inspection systems in hazardous environments. Future developments can enhance the robot's movement control, simplify deployment processes, and improve usability, making it an even more effective tool for RPV inspection and other industrial applications.

APPENDIX-A: FRONT COVER AND EDGE

Thesis Book Edge



Front Cover Page

DESIGN AND DEVELOPMENT OF AUTONOMOUS INSPECTION ROBOTIC SYSTEM FOR REAL-TIME DETECTION OF METAL FAILURE USING DEEP LEARNING

Shahmir (GL)	**(20MTE034)**
Abdul Qayoom	**(20MTE019)**
Kashif Ali	**(20MTE026)**

Bachelor of Engineering (B.E)
Mechatronic Engineering

MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO

2024

REFERENCES

[1] Zhang, H. Y., Lin, W. M., & Chen, A. X. (2018). Path planning for the mobile robot: A review. *Symmetry, 10*(10). https://doi.org/10.3390/sym10100450

[2] Li, Z., Wei, X., Hassaballah, M., Li, Y., & Jiang, X. (2024). A deep learning model for steel surface defect detection. *Complex and Intelligent Systems, 10*(1), 885–897. https://doi.org/10.1007/s40747-023-01180-7

[3] Ajmi, C., Zapata, J., Elferchichi, S., Zaafouri, A., & Laabidi, K. (2020). Deep Learning Technology for Weld Defects Classification Based on Transfer Learning and Activation Features. *Advances in Materials Science and Engineering, 2020*. https://doi.org/10.1155/2020/1574350

[4] Eng, 言語 :, Ieee, 出版者 :, Liangzhi, L., Ota, K., & Dong, M. (2017). Deep Learning for Smart Industry: Efficient Manufacture Inspection System with Fog Computing. *IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS: Vol. XX*. http://hdl.handle.net/10258/00009957

[5] Proceedings, 2019 24th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA): Paraninfo Building, University of Zaragoza, Zaragoza, Spain, 10-13 September, 2019. (2019). IEEE.

[6] Gonzalez, R., Mahulea, C., & Kloetzer, M. (2015). A Matlab-based Interactive Simulator for Mobile Robotics. https://doi.org/10.0/Linux-x86_64

[7] SANDHYA RANI Assistant Professor, B., & Student, M. (2016). Implementation of Smart Home Control by Using Low Cost Arduino & Android Design. *International Journal for Research Trends and Innovation (www.ijrti.org)* (Vol. 1). www.ijrti.org

[8] Purnomo, T. W., Danitasari, F., & Handoko, D. (2023). Weld Defect Detection and Classification based on Deep Learning Method: A Review. *Jurnal Ilmu Komputer Dan Informasi (Journal of Computer Science and Information), 16*(1), 77–87. https://doi.org/10.21609/jiki.v16i1.1147

[9] Agarwal, D., & Bharti, P. S. (2020, November 6). MATLAB Simulation of Path Planning and Obstacle Avoidance Problem in Mobile Robot using SA, PSO and FA. 2020 IEEE International Conference for Innovation in Technology, INOCON 2020. https://doi.org/10.1109/INOCON50539.2020.9298313

[10] Guruji, A. K., Agarwal, H., & Parsediya, D. K. (2016). Time-efficient A* Algorithm for Robot Path Planning. *Procedia Technology, 23*, 144–149. https://doi.org/10.1016/j.protcy.2016.03.010

[11] Somasundar, A. V. S. S., & Yedukondalu, G. (2018). Robotic path planning and simulation by jacobian inverse for industrial applications. *Procedia Computer Science, 133*, 338–347. https://doi.org/10.1016/j.procs.2018.07.042

[12] Abbadi, A., & Matousek, R. (n.d.). PATH PLANNING IMPLEMENTATION USING MATLAB.

[13] Zhang, Y., An, J.-P., & Chen, P. (2012). Research of Hybrid Programming with C#.net and Matlab. *Physics Procedia, 24*, 1677–1681. https://doi.org/10.1016/j.phpro.2012.02.247

[14]https://www.mesta-automation.com/how-to-write-an-hmi-with-c-and-wpf-part-1-of-x/

[15] https://openautomationsoftware.com/knowledge-base/overview-net-wpf-hmi/

[16]https://www.c-sharpcorner.com/UploadFile/e46423/arduino-control-using-a-windows-presentation-foundation-(wpf/

DESIGN AND DEVELOPMENT OF AUTONOMOUS INSPECTION ROBOTIC SYSTEM FOR REAL-TIME DETECTION OF METAL FAILURE USING DEEP LEARNING

ORIGINALITY REPORT

8%	6%	4%	3%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	muet.edu.pk Internet Source	3%
2	"Innovation, Engineering and Entrepreneurship", Springer Science and Business Media LLC, 2019 Publication	<1%
3	www.coursehero.com Internet Source	<1%
4	1library.org Internet Source	<1%
5	Submitted to Lourdes Hill College Student Paper	<1%
6	Submitted to Visvesvaraya Technological University, Belagavi Student Paper	<1%
7	jiki.cs.ui.ac.id Internet Source	<1%

8	Submitted to University of Bristol Student Paper	<1 %
9	jobs.utah.gov Internet Source	<1 %
10	Tavares, Miguel Pinheiro. "Path Planning for Different Types of Robots in Docking and Obstacle Avoidance", Universidade do Porto (Portugal), 2024 Publication	<1 %
11	bresdel.com Internet Source	<1 %
12	export.arxiv.org Internet Source	<1 %
13	Submitted to muetjamshoro Student Paper	<1 %
14	www.slideshare.net Internet Source	<1 %
15	diposit.ub.edu Internet Source	<1 %
16	usermanual.wiki Internet Source	<1 %
17	www.liangzhili.com Internet Source	<1 %
18	www.verifiedmarketresearch.com Internet Source	<1 %

19	Submitted to University of Westminster Student Paper	<1 %
20	openaccess.cag.edu.tr Internet Source	<1 %
21	proceedings.asmedigitalcollection.asme.org Internet Source	<1 %
22	scholar.ufs.ac.za Internet Source	<1 %
23	David Ho, Kurt Overley, Lee Short, H.A. Krieger. "Interpolating a topographical map of the ocean floor", Mathematical Modelling, 1986 Publication	<1 %
24	cse2.uta.edu Internet Source	<1 %
25	iaac-aeic.gc.ca Internet Source	<1 %
26	www.morrisriedel.de Internet Source	<1 %
27	www.scribd.com Internet Source	<1 %
28	"Proceedings of the 11th National Technical Seminar on Unmanned System Technology 2019", Springer Science and Business Media LLC, 2021	<1 %

Publication

29	Bhaveshkumar C. Dharmani, Suman Lata Tripathi. "Intelligent Circuit and Systems for SDG3-Good Health and Well-Being - Proceedings of the International Conference on Intelligent Circuits and Systems (ICICS 2023), October 12-13, 2023, Lovely Professional University, India", CRC Press, 2024 Publication	<1 %
30	Wang, Hanyin. "Enhancing Automated Diagnostics in Critical Care Harnessing the Power of Artificial Intelligence with Heterogeneous Health Data", Northwestern University, 2024 Publication	<1 %
31	udsspace.uds.edu.gh Internet Source	<1 %
32	Submitted to Asia Pacific University College of Technology and Innovation (UCTI) Student Paper	<1 %
33	Neerendra Kumar, Zoltan Vamossy, Zsolt Miklos Szabo-Resch. "Robot path pursuit using probabilistic roadmap", 2016 IEEE 17th International Symposium on Computational Intelligence and Informatics (CINTI), 2016 Publication	<1 %

34	core.ac.uk Internet Source	<1%
35	kipdf.com Internet Source	<1%
36	link.springer.com Internet Source	<1%
37	Ramon Gonzalez, Cristian Mahulea, Marius Kloetzer. "A Matlab-based interactive simulator for mobile robotics", 2015 IEEE International Conference on Automation Science and Engineering (CASE), 2015 Publication	<1%
38	Ton Duc Thang University Publication	<1%
39	Xin Chen, Zhuo Li, Chao Tang, Shenglong Xiao, Ying Chen. "Person re-identification in the edge computing system: A deep square similarity learning approach", Concurrency and Computation: Practice and Experience, 2019 Publication	<1%

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	Off		